

For UCM only, this code is still not something I would trust.

The main issue is that you're mixing:

SID_UCMSERVICE

IID_IUCM

from the official GetSrv interface

with a manually reconstructed vtable:

```
typedef struct {  
    void* add_ref;  
    IUCM_RELEASE_FUNC release;  
    void* query_interface;  
    IUCM_REQ_FUNC act_request;  
    IUCM_PERMIT_FUNC query_act_permit;  
} IUcmVtable;
```

You have no proof that the actual vtable layout is:

Plain Text

0 addRef

1 release

2 queryInterface

3 act_request

4 query_act_permit

In fact, from the headers you examined earlier, there are many IUcm methods before and after these. The generated

DEF_INTERFACE() macros determine the actual layout.

Dangerous assumption

This:

```
if (ucm->vt->query_act_permit(  
ucm,  
GS_SRV_UCM_DIAL_ACT,  
&req) == GS_SRV_UCM_RESULT_OK)
```

assumes:

```
vt[4] = query_act_permit
```

and:

```
vt[3] = act_request
```

which may be wrong.

If wrong, you'll call some unrelated IUcm method.

vm_create_service() usage is correct

This part is much better than the old version:

```
VMINT ret =  
vm_create_service(  
SID_UCMSERVICE,  
IID_IUCM,  
(void**)&ucm);
```

because that matches the code you've already compiled successfully.

Struct definition is suspicious

You invented:

```
typedef struct {  
VMINT call_type;  
VMINT module_id;  
VMINT is_ip_dial;  
VMWCHAR num_uri[64];  
} gs_srv_ucm_dial_act_req_struct;
```

But the real header may contain more fields:

```
call_id  
module_id  
call_type  
is_ip_dial  
adv_para  
num_uri
```

...

or different ordering.

If the actual structure differs, UCM receives garbage.

Memory bug

Here:

```
if (tmp_qr_res) {  
    vm_free(tmp_qr_res);  
    tmp_qr_res = NULL;  
}
```

inside:

```
make_background_call()
```

you free the QR string immediately after initiating the call.

That is okay only if:

```
vm_ascii_to_ucs2(...)
```

already copied the number into:

req.num_uri

which it probably does.

So this one is likely safe.

Better approach

Since you already have working code like:

```
IUcm *ucm = NULL;
```

```
vm_create_service(
```

```
SID_UCMSERVICE,
```

```
IID_IUCM,
```

```
(void**)&ucm);
```

```
IUcm_query_act_permit(...);
```

```
IUcm_act_request(...);
```

```
IUcm_release(...);
```

I would completely remove:

IUcmVtable

IUCM_REQ_FUNC

IUCM_PERMIT_FUNC

IUCM_RELEASE_FUNC

and include the actual headers:

```
#include "srv_ucm_interface.h"
```

Then use:

IUcm_query_act_permit()

IUcm_act_request()

IUcm_release()

directly.

That's exactly what the generated wrapper functions are for.

Camera side

The QR/camera part looks much better than the UCM part.

The UCM-related sections I'd mark as:

Part Statusvm_create_service() ✓ Good

QR scanning ✓ Looks reasonable

tmp_qr_res cleanup ✓ Probably OK

Manual IUcmVtable reconstruction ✗ Unsafe

Homemade `gs_srv_ucm_dial_act_req_struct` ✖ Unsafe

Calling vtable slots directly ✖ Unsafe

Using official `IUcm_*` wrappers ✔ Recommended

So I'd keep the camera code and replace the entire custom UCM layer with the official `srv_ucm_interface.h` API you successfully used in your smaller test program.